LMA Group Project 2026 · Mid-submission report · 31 October 2026

Stay on Task: Can a small classifier correct local agents' drift?

Shuban Biswas 2024111017 **Ch Pavan Harshit** 2025701057 **Swayam Goyal** 2024111012 **Aaditya Sharma** 2019113009

Team Stay on Task · Code, data and results: `https://github.com/ch-pavan/lma-grp-project/tree/main`

Abstract. Small agents that run locally and act through tools often lose track of a request, make up values, or claim work they never did. The usual guard is a language-model judge on every step, which is slow and costly. We test whether Laya, a 0.4B classifier finetuned for typed decisions, can judge an agent's steps better than a prompted 9B judge (the *SLM judge*) given the same state, at a fraction of the cost.

At mid-term we have built and run:

- a planner, doer and judge harness on a cleaned BFCL multi-turn split
- three baseline systems run in full on the test set, plus pilots of two more
- recorded runs of three agent models
- LLM labels for every validation step under guide v2, which the 200-step human check scored at kappa 0.53 and 0.62; a rewritten guide (v3) reaches 0.84 and 0.71 on a fresh sample, so far only on the 200 checked steps
- two finetuned Laya heads

On validation, finetuned Laya ranks bad tool calls far better than the SLM judge (AUROC 0.91 against 0.61 for the judge's yes/no answers, on the replay step labels). Against the LLM annotators, with the judge on its harness prompt, Laya's F1 lead on off-track tool calls is +7 to +9 points, with confidence intervals that include zero. For telling whether a turn is finished, Laya's AUROC is 0.79 (0.53 as is); its F1 beats the SLM judge's 0.41 only at threshold 0.5 (0.53), not at the threshold chosen on dev (0.36). Each Laya check costs one forward pass, where the SLM judge writes about 1,500 to 2,400 tokens. Scored on the same labels, Laya beats plain rules on both classes of the primary metric (primary F1 0.18 to 0.19 against 0.06 to 0.07; no paired CI yet). The proposal's primary bar is a gain of 9 F1 points on the mean of off-track and false-claim F1. On validation the gap is only -1.0 to $+2.0$. Under the proposal's scoring, Laya trails the judge slightly on off-track F1 over all steps (its "not done" answers count as false claims, so it never marks a closing message off track), and both are near zero on false claims, which are rare on validation and which Laya has no head for. The plan to the final submission therefore focuses on test passes sized for false claims, a false-claim head, and the test-set evaluation.

Contents

1 Introduction **2**

2 Team and roles **3**

3 Approach **3**

4 Progress against the proposal **4**

5 Datasets **5**

5.1 Benchmarks . . . 5

5.2 Recorded agent runs . . . 5

5.3 Labels and class balance . . . 5

5.4 Annotation . . . 6

5.5 Laya training sets . . . 6

6 Evaluation protocol **6**

7 Baseline experiments **7**
7.1 Choice of agent model . . . 7
7.2 Online judges on the test set . . . 8
7.3 Off-the-shelf judges, offline . . . 8

8 Results: finetuned Laya **9**
8.1 Offline . . . 9
8.2 Live on the agent . . . 10
8.3 Primary metric on validation . . . 10

9 Ablations **11**

10 Annotation quality check **12**

11 Worked example of a failure **13**

12 Challenges and mitigation **14**

13 Changes from the proposal **14**

14 Timeline to the final submission **15**

15 Repository and reproduction **16**

Table 1. Where each required part of the mid-submission is.

Required	Section
Progress	Section 4 (status of every proposal item), Section 13 (changes from the proposal)
Datasets being used	Section 5
Baseline experiments	Section 7
Ablations	Section 9
Results from the experiments run	Sections 7, 8 and 10
Challenges faced and mitigation plans	Section 12
Timeline for the final submission	Section 14
Updated GitHub with all the code	Section 15: `https://github.com/ch-pavan/lma-grp-project/tree/main`

1 Introduction

Problem. The target user wants an agent that runs on their own machine, private and cheap, and acts on files, messages, posts, bookings and trading accounts. Small local models drift partway through such tasks in four ways:

- they lose track of the request
- they fill in values nobody gave
- they report success after a failed call
- they make changes nobody asked for

A large judge on every step is too slow, and a small model judging itself is weak.

Research question. Can a finetuned small classifier, given the same state as an SLM judge, detect off-track steps and false claims better than that judge? And what does a system built around it do to task accuracy and cost?

Table 2. Hypothesis and falsification, as in the proposal.

Claim	Falsified if
Primary. Finetuned Laya beats the SLM judge at catching off-track steps and false claims.	The gain in mean F1 (off track, false claim) is under 9 points, or its 95% CI includes zero. A gain of 5 to 9 points is reported but does not confirm the hypothesis.
Task. Our system completes more tasks than no judge and about as many as the SLM-judge system, for a fraction of its judge tokens.	Tested only if the primary holds. It fails if the system does not beat no judge with a CI above zero, or if the lower end of its 95% CI against the SLM-judge system is more than 5 points down.

Key findings at mid-term.

- **Laya ranks bad steps much better than the SLM judge offline** (AUROC 0.91 against 0.61 for the judge's yes/no answers, on the replay labels the head was trained on), at near-zero cost. Against the LLM annotators its F1 lead on off-track calls is +7 to +9 points, not yet significant (Section 8).
- **Laya beats plain rules on both classes of the primary metric** when both are scored on the same LLM labels: primary F1 0.18 to 0.19 against 0.06 to 0.07, no paired CI yet. On made-up values, which Laya does not output, the rules score F1 0.44 to 0.52 (Section 9).
- **A judge of single steps cannot see omissions,** which are about half of all failures, so we added a check at the end of each turn (Section 7.1).
- **Block-and-warn corrections do not help even with a perfect judge,** and they hurt with the SLM judge. Rollback plus the turn-end check moved task accuracy with a perfect (oracle) judge on the weaker Qwen3.5 4B agent; no deployable judge has moved it yet (Section 9).
- **After one guide rewrite, the LLM annotators reach kappa 0.84 and 0.71 against the human check** on a fresh sample (reweighted; 0.71 is 0.69 unweighted): past the 0.7 bar only after reweighting, and in the proposal's "label more steps" band (Section 10).
- **The primary metric cannot be measured on validation yet,** because there are too few false claims and Laya has no false-claim head. The test passes are sized for it (Section 12).

2 Team and roles

Every member worked across design, experiments and writing. Table 3 gives each person's main ownership.

Table 3. Roles and main work.

Member	Role	Main work
Swayam Goyal	Annotation check, guide v3, report	Cross-checked the LLM annotations: labelled the 200-step check sample blind, with Aaditya. Rewrote the annotation guide (v3) from the check results, built the fresh check sample, wrote this report.
Aaditya Sharma	Annotation check, error analysis	Cross-checked the LLM annotations: labelled the 200-step check sample blind, with Swayam. Analysed where the LLM annotations differ from the human check by error type; reviewed the guide and the proposal.
Ch Pavan Harshit	Agent runs, LLM annotation, Laya, referee	Ran the Qwen3.5 9B agent on Turing and the LLM annotation runs, finetuned Laya's two heads, ran the live checks and the primary-metric analysis; refereed the annotation check.
Shuban Biswas	Harness, baselines, second pass	Built the harness and the replay oracle; ran the Qwen3 8B baselines on BFCL test and AgentDojo and the Qwen3 8B and Qwen3.5 4B validation runs, the correction-action study, and the offline scorer for off-the-shelf judges; did the blind second pass on the contested check steps.

All four of us wrote and revised the proposal.

3 Approach

Agent. For each user turn, a planner writes a short plan and a doer carries out the steps through tool calls. A judge checks each step before it runs, so a flagged call never executes. Both agent roles use Qwen3.5 9B (4-bit AWQ, reasoning off) served with vLLM.

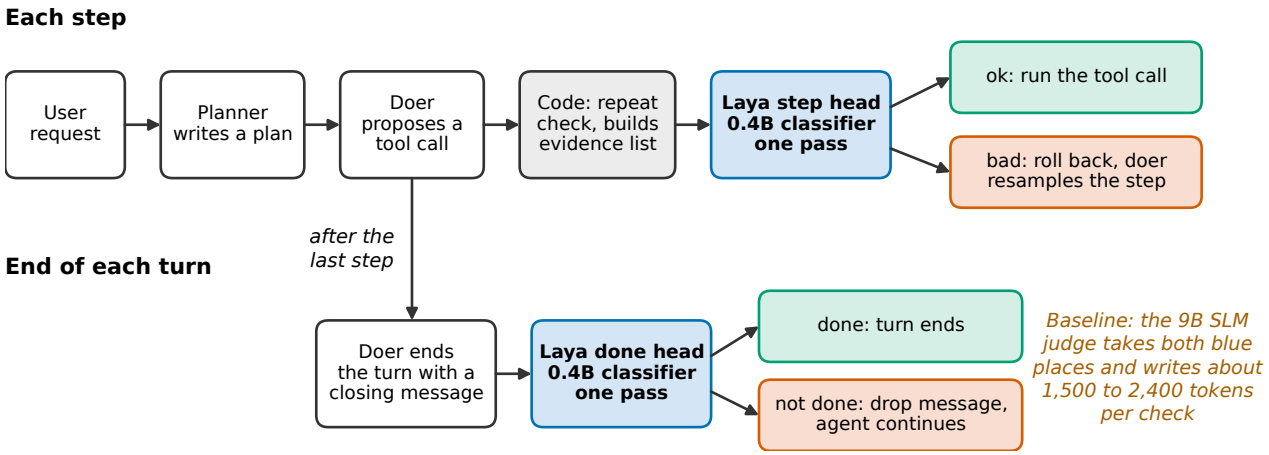


Figure 1. Our system. The blue boxes are the two Laya checks. In the baseline, the 9B SLM judge sits in both places and reasons before each answer.

State given to the step judge. The Laya step head and the SLM judge get the same input. Code builds it, up to 1,024 tokens:

- the proposed call or closing message, placed first
- the current and earlier user requests
- the turn plan
- the last three calls, with their outputs cut short
- an *evidence list*: for every argument value, where it appears (request, tool output, tool docs, environment, composed, arithmetic, or nowhere)

The done check instead gets the closing message, the user requests, and every call of the turn with its output cut

short.

Questions. Code settles what needs no model: an exact repeat of a failed call. Laya answers typed questions, each with a fixed set of options and a confidence:

- **Step head, on every tool call:** is this a bad step? Bad covers three cases:
  - off track: the turn can no longer pass
  - an unneeded irreversible call
  - a made-up value on an irreversible call
- **Done head, at every turn end:** is the latest request really finished, done or not done? A typed head (done, missing action, should ask the user, unsupported claim) is planned.

Actions. A bad step is rolled back and the doer resamples it, at most twice. A "not done" answer drops the closing message and tells the agent that part of the request may be unfinished, at most twice per turn.

4 Progress against the proposal

Table 4. Status of each item of the revised proposal (`proposal/proposal.md`) at mid-submission.

Proposal item	Status
Harness: planner, doer, judge, fixed actions, rollback	**Done.** 20-step cap per turn. Correction modes: block and warn, rollback, deep rollback, vote of 3, done check.
BFCL multi-turn, cleaned, split by scenario, empty-turn check	**Done.** 74 train, 15 validation, 100 test scenarios; 11 unsound scenarios dropped. Gold replay passes 755 of 756 tasks.
AgentDojo benign tasks as an outside check	**Done** for no judge, rules and the SLM judge on the earlier Qwen3 8B agent (97 tasks $\times$ 3 runs); to be rerun with the Qwen3.5 9B.
Natural agent runs for training	**Done for the 9B:** 3 passes over the train scenarios (888 tasks) and 120 validation tasks. The planned Qwen3.5 4B pass was replaced by recorded Qwen3 8B steps (356 tasks).
Directed runs with a stronger model (Qwen3.8 27B)	**Dropped** (Section 13). Training pools natural mistakes from two agent models (Qwen3.5 9B and Qwen3 8B).
Baselines: no judge, rules, SLM judge, Laya as is, Granite Guardian	**Done on test with the earlier Qwen3 8B agent and Qwen3 8B judge** for no judge, rules and the SLM judge (1,200 runs each); the proposal's agent and judge are the Qwen3.5 9B, so these are to be rerun for the task claim. Laya as is was stopped after 133 runs and Granite Guardian ran as a 20-task pilot, both because they block most steps (Section 7.2).
Offline judges: Kev, LettuceDetect and others	**Done on validation** (Section 7.3). Still to run offline: Granite Guardian, Jev on the full set, and Gemma 4 12B as a stronger reference judge.
Finetuned Laya	**Done for two heads,** step and done (Section 8). A false-claim head is planned.
LLM annotation by two annotators	**Done for all 1,772 validation steps under guide v2.** Guide v3 so far only on the 200 check steps; the full relabel is deferred.
Human check of the LLM labels on a stratified sample	**200 validation steps checked,** guide rewritten once; the decision rule asks for more steps, which come from the test runs (Section 10).
Check of each automatic label rule by the annotators (50 steps per option, 80% bar)	**Not done as specified.** The oracle's off-track rule agrees poorly with the annotators (kappa 0.04 to 0.15) but still supplies most step-head positives (Section 12).
Gate 1: at least 300 training examples per error class	**Not met.** With the 9B and 8B steps from training scenarios pooled (automatic labels), off track (310) and false claim (372) pass 300; unneeded irreversible calls (217) and made-up values (at most 143) do not.
Test labels and the primary metric on test	**Not yet.** Starts after the GPU budget resets on 1 November.
Confidence-gated escalation to the SLM judge; calibration on 200 annotator-labelled validation steps	**Not yet,** after the test labels.
Laya finetuned alone and the full system, online on test	**Not yet** (Phase 4 of the plan).

Proposal item	Status
Any-flag F1 on AgentProcessBench and AgentHallu	**Not yet;** future work if time allows.
Rules and Laya scored on the same LLM labels	**Done** (Table 15).
Step head retrained without the oracle "breaks task" label, 3 seeds of each variant	**Running** on Ada (RTX 2080 Ti, fp16), results for the final if not ready.
Done-head threshold chosen by grouped cross-validation on train	**Running** on Ada, results for the final if not ready.

5 Datasets

5.1 Benchmarks

BFCL multi-turn (gorilla commit `6ea5797`):

- **Size.** 200 scenarios in four versions: base, missing parameter, missing function and long context.
- **Domains.** Eight simulated apps: files, messages, posts, tickets, travel, trading, car and maths.
- **Cleaning.** 11 scenarios with unsound answer keys are dropped.
- **Split by scenario.** 74 train, 15 validation, 100 test. The test scenarios are never used for training, tuning or prompt choice.
- **Pass rule.** BFCL's own state checker plus our empty-turn check: on a turn whose correct answer is to call nothing, any state change fails it.

AgentDojo benign user tasks (agentdojo 0.1.35, suites v1.2.2):

- **Size.** 97 tasks across workspace, travel, banking and Slack, with no injections.
- **Pass rule.** Each task's own utility check.
- **Use.** Outside check on task accuracy only: no training or tuning on it.

5.2 Recorded agent runs

Table 5. Recorded agent runs (in `runs/raw/` and `shuban_findings/finetune_data/`).

Run	Tasks	Steps
Qwen3.5 9B, validation, reasoning off	120	1,277 calls, 495 turn ends
Qwen3.5 9B, validation, reasoning on	120	776 calls, used in the step check's validation set
Qwen3.5 9B, train	888	8,806 calls, 3,737 turn ends
Qwen3 8B, train and validation scenarios	356	3,824 steps, plus 3,635 gold replay steps

5.3 Labels and class balance

A replay oracle labels every step automatically. After each call, it replays every order-preserving subset of the turn's reference calls (up to 8) as the rest of the turn. The first call after which no ending passes is off track.

Table 6. Class balance of the automatic labels (Qwen3.5 9B runs).

Label	Unit	Validation	Train
Off track	tool call	63 of 1,277 (4.9%)	221 of 8,806 (2.5%)
False claim (success words while the state is wrong)	closing message	50 of 495 (10.1%)	281 of 3,737 (7.5%)
Turn not done (wrong end state, or a reference function never called)	turn end	111 of 495 (22.4%)	672 of 3,737 (18.0%)

How we handle the imbalance.

- **Metrics:** we report per-class F1, precision, recall and AUROC, never accuracy.

- **Training:** finetunes use balanced class weights. The step head's threshold is chosen on held-out dev scenarios; the done head is reported at both 0.5 and its dev threshold (Section 8).
- **Gate 1 is not met.** With the 9B and 8B steps from training scenarios pooled, off track (310) and false claim (372) reach 300, but unneeded irreversible calls (217) and made-up values (at most 143) do not. The turn-end "not done" label (672 in 9B train) is better balanced, but it is not a proposal class.

5.4 Annotation

Two LLM annotators from different model families labelled all 1,772 validation steps from a written guide, through `analysis/annotate.py`, without seeing the oracle labels or the BFCL verdict:

- **Annotator A:** DeepSeek V4.1 Flash
- **Annotator B:** GLM-5.3-Flash (the revised proposal names GLM-5.3; we used its Flash variant)

Swayam and Aaditya cross-checked these labels on a 200-step sample (Section 10).

What each step gets:

- **Tool call, Q1:** on track or off track.
- **Tool call, Q2:** none, made-up value or false claim.
- **Turn end:** one typed answer, from done, missing action, should ask the user, and unsupported claim.

Section 10 gives the quality check of these annotations.

5.5 Laya training sets

Table 7. Laya training sets, split by scenario.

Head	Train	Dev (threshold, epoch)	Validation
Done check	3,341 turn ends, 66 scenarios	396 turn ends, 8 held-out train scenarios	495 turn ends
Step check (9B and 8B steps pooled; validation also has the 9B reasoning-on run)	8,847 calls, 408 bad	1,975 calls, 55 bad	2,427 calls, 186 bad

6 Evaluation protocol

- **Detection, offline:** per-class F1, precision, recall and AUROC on recorded steps.
- **Primary metric:** the mean of off-track F1 and false-claim F1, computed against the annotations.
- **Task accuracy, online:** the share of tasks passing BFCL's checker plus the empty-turn check.
- **Cost:** judge tokens and seconds per task.
- **Confidence intervals:** 95%, from a bootstrap over scenarios, never over tasks, since tasks from one scenario are correlated.
- **Comparing systems:** differences use a paired bootstrap on the same tasks. Secondary comparisons are Holm-corrected.

Table 8. Systems compared.

Code	System
S1	**No judge:** the planner and doer alone.
S2	**Plain rules:** a value with no evidence is made up; a success word with no successful call is a false claim; a call not in the plan is drift; every irreversible call asks the user.
S3	**SLM judge:** the agent model as judge, reasoning on, given the same state and guide as Laya.
S4a / S4b	Laya typed-decisions 0.4B as is / Granite Guardian 3.2 3B A800M.
Ours	**Finetuned Laya:** step head and done head, with rollback and the done check.
Oracle	**Ceiling, not deployable:** flags exactly the steps the replay oracle marks as wrong.

7 Baseline experiments

7.1 Choice of agent model

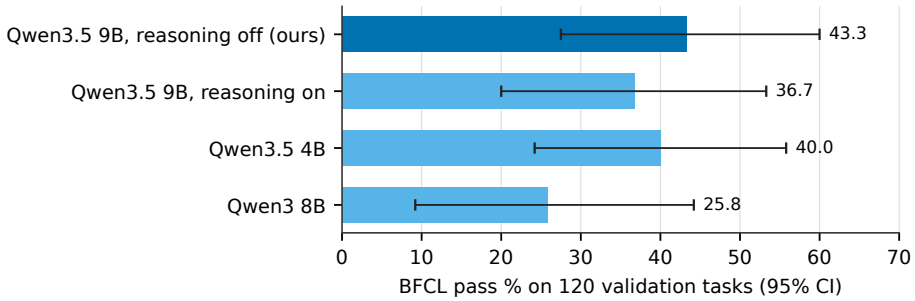


Figure 2. Agent models on the same 120 validation tasks. The Qwen3.5 9B with reasoning off is our agent.

Figure 2 compares four agent setups:

- **The model generation matters more than size.** Qwen3.5 4B passes 40.0%, against 25.8% for the older Qwen3 8B.
- **The 9B passes 43.3%** on validation and 52.7% [44.8, 60.5] on the 888 train tasks.
- **Reasoning on does not help.** It is 6.6 points lower and 2.5× slower, so the agent runs with reasoning off.

Table 9. Why the 9B agent fails, by the first unfinished turn of each failed validation task.

Failure type	Meaning	Share
Stopped early	ended the turn with no success claim, often after a fixable error	26.5%
Wrong step	a call that made the right end state unreachable	23.5%
Acted instead of asking	changed state where the right move was to ask	22.1%
Claimed done	said it was done while the state was wrong	17.6%
Asked needlessly	asked the user although the request was answerable	8.8%
Checker only or step cap		1.5%

About half of all failures are unfinished turns: stopped early, claimed done, or asked needlessly. A judge of the proposed step cannot see something that never happened. **This finding is why we added the done check at turn end.**

7.2 Online judges on the test set

Table 10. Baselines on the BFCL test set: Qwen3 8B agent, 100 scenarios × 4 versions × 3 runs.

System	Runs	Pass % [95% CI]	vs S1 [95% CI]	s / task	Judge tok. / task
S1 no judge	1,200	15.9 [11.7, 20.5]		93	0
S2 plain rules	1,200	7.6 [4.4, 11.2]	−8.3 [−11.8, −5.2]	108	0
S3 SLM judge	1,200	17.4 [12.8, 22.3]	+1.5 [−0.6, +3.8]	202	11.8k
S4a Laya as is	133 (stopped)	0 (upper bound 2.2)		336	
S4b Granite Guardian	20-task pilot	0			

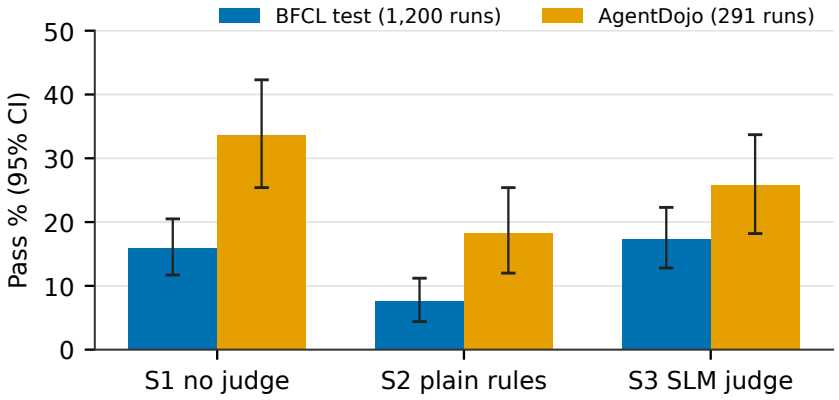


Figure 3. Pass rates on BFCL test and AgentDojo, with 95% confidence intervals.

- **Rules hurt.** They block good calls: their made-up-value rule flags 1.8% of gold calls.
- **Laya as is blocks almost everything.** It blocked 87.6% of judged steps, so every run hit the step cap.
- **Granite Guardian also blocks almost everything.** It called all 169 calls it saw risky.
- **The SLM judge is the only system that does not hurt on BFCL.** It gains most on the missing-function (+7.3) and missing-parameter (+3.0) versions, but overall it is within noise, at twice the time per task.
- **Out of domain, the SLM judge loses.** On AgentDojo it drops 7.9 points (Figure 3).

7.3 Off-the-shelf judges, offline

Table 11. Primary score (mean F1 of off-track and false-claim steps) on the 1,772 validation steps of the 9B agent, against the oracle and the guide v1 LLM annotations.

Judge	vs oracle	vs Annotator A	vs Annotator B
Plain rules	0.07	0.10	0.11
Laya as is	0.11	0.21	0.16
Kev 0.8B	0.15	0.12	0.07
LettuceDetect (false claim only)	0.18	0.22	0.15

Every confidence interval overlaps, so none is reliably ahead, and all score low (primary F1 0.07 to 0.22). These scores use the guide v1 labels; the comparison of rules and Laya on the same v2 labels is in Table 15.

Laya as is. On the Qwen3 8B validation steps against the oracle labels, we swept it across 4 state formats, 8 question wordings, thresholds and a logistic head. Its AUROC stayed at 0.39 to 0.67. It keeps the start of its input and cuts the end, so on long states it never saw the step it was judging. Putting the step first fixed the cut but added no signal.

Jev. On the same steps, the hosted Jev classifier was the only off-the-shelf model with real signal, and only on closing messages (AUROC 0.81). **Finetuning is required.**

8 Results: finetuned Laya

8.1 Offline

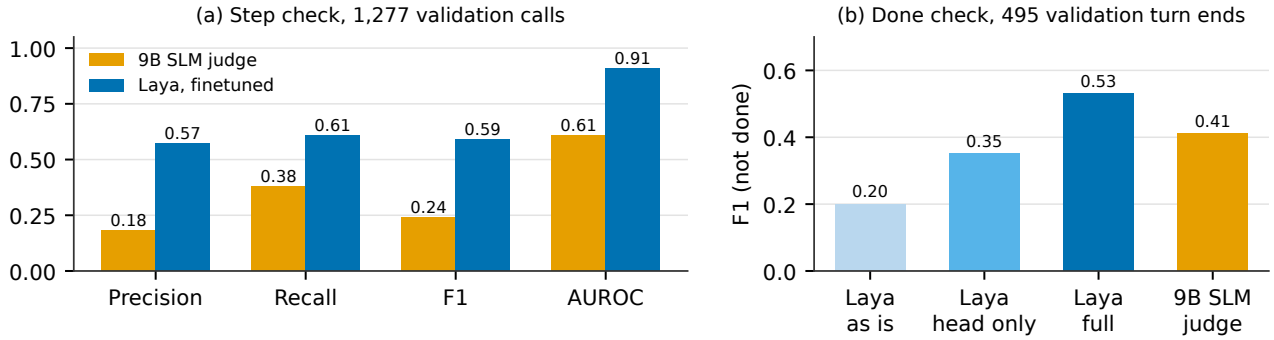


Figure 4. Finetuned Laya against the 9B SLM judge on the same validation steps. (a) Step check on tool calls. (b) Done check at turn end; Laya full is at threshold 0.5, and at its dev-chosen threshold its F1 is 0.36.

Both heads use a full finetune with balanced class weights on one RTX 6000 Ada: 4 epochs and 16 minutes for the step check, and 4 epochs and 5 minutes for the done check.

Table 12. Offline detection on validation: the step check on 1,277 calls (109 bad); the done check on 495 turn ends (positive class: not done). Step labels are the replay step labels the step head was trained on; the 9B SLM judge here used the v1 annotator prompt, and 83 of its 1,277 call answers and 20 of its 495 turn-end answers did not parse and count as not flagged. Done-check F1: the full finetune at threshold 0.5 (at its dev-chosen threshold, 0.98, it gives 0.36, below the SLM judge's 0.41); the as-is and head-only rows at their dev thresholds (0.49 and 0.54).

Check	Checker	Precision	Recall	F1	AUROC
Step	9B SLM judge (232 flags)	0.18	0.38	0.24	0.61
Step	**Laya, finetuned (116 flags)**	**0.57**	**0.61**	**0.59**	**0.91**
Done	Laya as is			0.20	0.53
Done	Laya, head only			0.35	0.59
Done	**Laya, full finetune**			**0.53**	**0.79**
Done	9B SLM judge			0.41	

The step-check gain is not just reading function names. On the 2,427 pooled validation calls (9B, 9B with reasoning on, and 8B), finetuned Laya reaches AUROC 0.90, against 0.57 for Laya as is.

- **On irreversible calls only,** Laya scores AUROC 0.74 against 0.50 for the rule "flag every irreversible call".
- **On reads only,** it scores 0.95 against 0.50.
- **It holds on every data source:** 0.91 on 9B steps, 0.83 with reasoning on, and 0.92 on 8B steps.

Fairer comparison. With the SLM judge on its harness prompt and the LLM annotators' labels, Laya's lead on off-track tool calls is +8.2 [−1.0, +13.5] (Annotator A), +7.2 [−3.0, +12.8] (B) and +9.2 [−0.6, +15.2] where both agree (Table 14). Over all steps, under the proposal's scoring, it trails slightly. The threshold-free AUROC is the most robust result; the F1 gaps depend on the labels, the prompt and the threshold.

Cost. The 9B SLM judge writes about 1,500 output tokens per call and 2,380 per closing message. Laya runs one forward pass of a 0.4B encoder.

8.2 Live on the agent

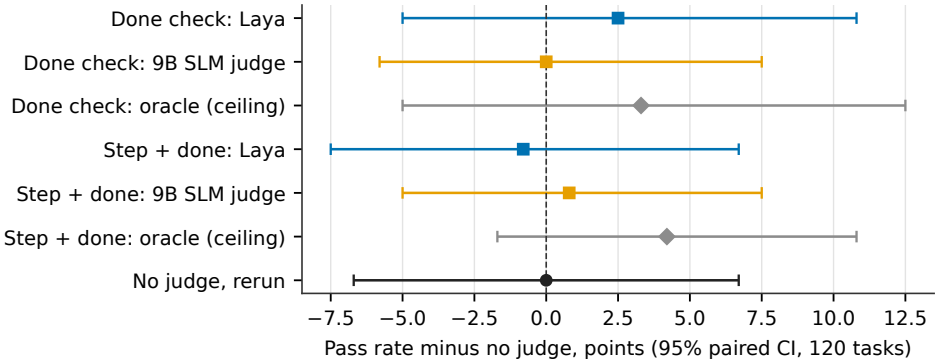


Figure 5. Live runs on the 120 validation tasks: pass rate minus no judge, with 95% paired confidence intervals.

Table 13. Live results on the 120 validation tasks.

System	Pass	vs no judge [95% CI]	Judge tok. / task	s / task
No judge	43.3%		0	207
Done check: **Laya**	**45.8%**	**+2.5 [−5.0, +10.8]**	≈0	234
Done check: 9B SLM judge	43.3%	+0.0 [−5.8, +7.5]	10,334	454
Done check: oracle	46.7%	+3.3 [−5.0, +12.5]	0	235
Step and done checks: Laya	42.5%	−0.8 [−7.5, +6.7]	≈0	
Step and done checks: 9B SLM judge	44.2%	+0.8 [−5.0, +7.5]		
Step and done checks: oracle	47.5%	+4.2 [−1.7, +10.8]	0	

- **The Laya done check scores 2.5 points higher than the SLM-judge done check** (not significant), at about zero judge tokens and half the time per task.
- **No live difference is significant.** Two identical no-judge runs disagree on 28 of 120 tasks.
- **There is little room on this agent.** A perfect checker adds only +4.2, against +9.2 on the weaker Qwen3.5 4B agent. A live gain therefore needs more seeds or a weaker agent.

8.3 Primary metric on validation

Table 14. Primary metric on validation: finetuned Laya against the 9B SLM judge, scored against the guide v2 LLM annotations (which the human check scored at kappa 0.53 and 0.62). CIs: paired bootstrap over scenarios. The proposal's scoring counts Laya's "not done" as a false claim and every closing message as on track for Q1, so Laya never marks a closing message off track. The last column scores tool calls only. The 9B judge used its harness prompt here; 33 of its 1,772 answers (1.9%) stayed unparsed and count as not flagged.

Labels	**Off track, all steps** (Laya / 9B)	**False claim** (Laya / 9B, positives)	**Primary,** Laya − 9B [95% CI]	**Off track, tool calls only** (Laya / 9B; Laya − 9B [95% CI])
Annotator A	0.30 / 0.34	0.07 / 0.05 (15)	−1.0 [−7.3, +3.2]	0.42 / 0.34; +8.2 [−1.0, +13.5]
Annotator B	0.27 / 0.29	0.10 / 0.05 (10)	+2.0 [−5.4, +7.1]	0.36 / 0.29; +7.2 [−3.0, +12.8]
Both agree	0.30 / 0.31	0.07 / 0.06 (6)	+0.5 [−6.7, +5.1]	0.39 / 0.29; +9.2 [−0.6, +15.2]

- **Off track, all steps:** Laya trails the 9B slightly (0.27 to 0.30 against 0.29 to 0.34), because under the proposal's scoring its done head only ever signals false claims, never an off-track closing message.
- **Off track, tool calls only:** Laya leads by +7 to +9 points, with intervals that include zero.
- **False claim:** both are near zero (Laya 0.07 to 0.10, the 9B 0.05 to 0.06), on only 6 to 15 positives, and Laya has no false-claim head yet.

- **The primary hypothesis is neither met nor falsified on validation.** Section 12 gives the plan.

9 Ablations

Table 15. Ablations run so far. The first three are the deeper ablations the TA asked for.

Ablation	Result
Correction action (Qwen3 8B, validation)	**Block and warn does not help even with a perfect judge** (oracle +2.5 [−1.7, +6.7]) and hurts with the SLM judge (18.3% against 25.8% for no judge). Rollback removes that damage (22.5 to 24.2%). A vote of 3 with the SLM judge does not help (20.8%, −5.0 [−12.5, +1.7]). **Rollback plus the done check moved results with the oracle** on Qwen3.5 4B (+9.2 [+2.5, +15.8]); on Qwen3 8B it is borderline (+5.8 [0.0, +12.5]). No deployable judge has moved results yet.
Finetuning gain per question	Done check: AUROC 0.53 as is, 0.59 head only, 0.79 full finetune. Step check: AUROC 0.57 as is against 0.90 finetuned (2,427 pooled validation calls).
Per question against rules	Scored on the same guide v2 labels as Table 14 (A / B). Off track over all steps: rules 0.14 / 0.12, Laya 0.30 / 0.27. On tool calls: rules 0.16 / 0.14, Laya 0.42 / 0.36. False claim: rules 0.00 / 0.00, Laya 0.07 / 0.10. Primary: rules 0.07 / 0.06, Laya 0.18 / 0.19. Laya beats the rules on off track and false claim (no paired CI yet); the rules flag 299 calls off track at precision 0.08 to 0.09. On made-up values, which Laya does not output, the rules score F1 0.44 / 0.52.
Agent model and reasoning	Qwen3 8B 25.8%, Qwen3.5 4B 40.0%, Qwen3.5 9B 43.3%. Reasoning on is 6.6 points lower and 2.5× slower.
Input format, Laya as is	4 state formats and 8 question wordings: AUROC 0.39 to 0.67. Putting the step first fixes the truncation but adds no signal.
How the done head is used	As a gate on every closing message: 37.5%. Only for the done check: 42.5%.
SLM judge prompt	Against Annotator A's labels, the harness judge prompt raises the 9B's call F1 from 0.28 to 0.34, which shrinks Laya's lead on calls from +14.5 to +8.2 points.
Annotation guide version	On the fresh sample, agreement with the human check rises from kappa 0.71 to 0.84 for Annotator A and from 0.66 to 0.71 for Annotator B. Over all 200 steps, which include the pilot that v3 was written from (optimistic): 0.53 to 0.87 and 0.62 to 0.71 (Section 10).

Running now (Ada, RTX 2080 Ti, fp16; results for the final if not ready):

- **Training labels.** The step head retrained without the oracle "breaks task" label (which failed the 80% agreement rule; 209 instead of 408 bad training calls), against 3 seeds of the current labels in the same setup.
- **Done-head threshold.** Chosen from out-of-fold scores on 5 scenario-grouped folds of train, so validation is never used to choose it.

Planned for the final submission (all offline and cheap):

- **State parts.** Drop the plan, the recent calls or the evidence list from Laya's input, and measure the effect on off-track detection. This tests our core state design, as the TA asked.
- **Escalation budget.** Send 0, 10, 20, 30 or 100% of uncertain steps to the SLM judge, and plot F1 against judge tokens.
- **Seeds.** Run three seeds for every finetune.
- **Question order.** Compare feeding the Q1 answer to Q2 with leaving it out.

10 Annotation quality check

The revised proposal requires the LLM annotation to be checked by people on a stratified sample, half flagged by the annotators and half not:

- **Checkers:** Swayam and Aaditya each labelled every sampled step blind, and Pavan refereed their disagreements. They saw neither the LLM labels nor the oracle labels. Checker-vs-checker agreement is measured on the raw sheets, before refereeing.
- **Sample:** two rounds of 100 validation steps, each with 50 calls and 50 turn ends.
- **Reweighting:** kappa is reweighted to the real mix of steps.
- **Decision rule:** below 0.6, rewrite the guide; between 0.6 and 0.8, label more steps. The pass bar is 0.7.

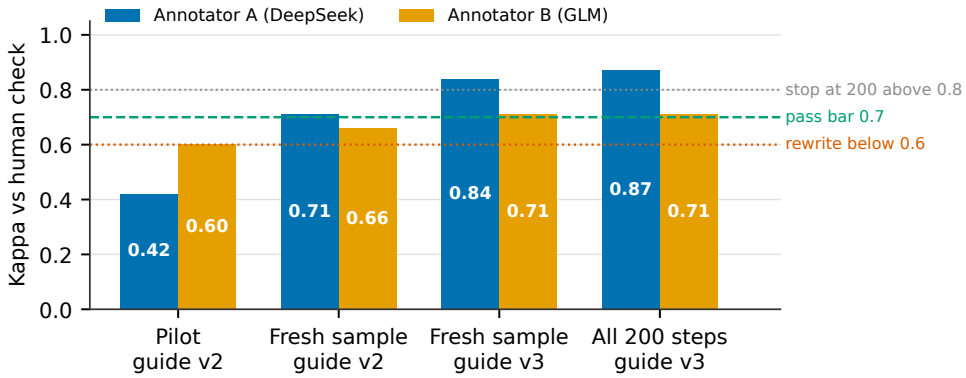


Figure 6. LLM annotators' agreement with the human check consensus (Cohen's kappa, reweighted to the real mix of steps). Between the 0.6 and 0.8 lines the rule is "label more steps"; 0.7 is the pass bar. The last pair includes the pilot that v3 was written from.

Table 16. Annotation check results (kappa, reweighted; every step, problem vs fine). *The fresh-sample consensus was labelled under the first v3 draft, so the v2-to-v3 rise partly reflects v3's rule changes.

Round	Checker vs checker	vs A	vs B	A vs B	Decision (weaker annotator)
Pilot, guide v2	0.89	0.42	0.60	0.66	below 0.6: rewrite the guide
Fresh sample, guide v2*	0.87	0.71	0.66	0.66	0.6 to 0.8: label more steps
Fresh sample, guide v3 (main result)	0.87	**0.84**	**0.71**	0.73	0.6 to 0.8: label more steps (B is 0.69 unweighted)
All 200 steps, guide v3 (optimistic)		0.87	0.71	0.76	includes the pilot v3 was written from

What guide v3 changed.

- **Done means every part.** Split the request into its parts; a turn is done only if every part has a successful call behind it.
- **Plans do not authorise changes.** A change listed in the plan but not asked for in the request is still unrequested.
- **Steps toward the goal are on track,** such as copying a file before renaming it.
- **Self-chosen values need asking.** If the agent picks a name for a change itself, it should have asked the user. This rule is kept but contested: both annotators and Shuban's second pass disagreed with it under v2.
- **The evidence list is trusted.**

Caveats. The fresh sample was drawn after v3 was written, so it is the fairer test, but the checkers labelled it with the first v3 draft. Two rules were reversed afterwards (a zero-amount fill counts as a change; deleting a file the request did not ask to delete is off track), and no fresh-sample label depends on them. On 7 contested pilot steps, Shuban's blind second pass sided with the annotators every time; scoring the pilot with his answers would lift its kappas from 0.42 and 0.60 to 0.57 and 0.78. Under the proposal's rule the result is in the "label more

steps" band, and the next 100 check steps come from the test runs. The weakest class is false claims at turn ends (kappa 0.38 to 0.46 on few positives), which matters because false claims are half of the primary metric. Guide v3 also lowered false-claim agreement on the fresh sample (A from 0.76 to 0.46, B from 0.49 to 0.38; on the pilot, B is at 0.00), while turn-end done-vs-not agreement rose (A 0.66 to 0.78, B 0.56 to 0.61). The next guide revision targets false claims.

11 Worked example of a failure

Task `multi_turn_base_93` (car domain, validation, failed).

1. **"How much fuel do I have?"** One correct call. Done.
2. **"Fill it to full, start the engine with START, check the tire pressure."**
   - **What the agent did:** tried to fill 50 gallons (error: over capacity), tried to start the engine (error: the doors must be locked, naming the unlocked doors), checked the tire pressure, then reported what it could not do.
   - **What it missed:** locking the doors and pressing the brake; the error named the unlocked doors.
   - **Pattern:** stopped early, an easy catch for a done check.
3. **"If tire pressure is below 40 psi, give me directions to the nearest tire shop."**
   - **What the agent did:** found the shop and told the user the address.
   - **What it missed:** it never set the navigation.
   - **Pattern:** the agent is confident, and "directions" has to be read as an action. Subtle cases like this are where an SLM judge with reasoning may still beat a 0.4B classifier. That is why the proposal keeps escalation to the SLM judge for uncertain steps.

No call in this task was harmful. Both failures are omissions, which a judge that only sees proposed steps cannot see.

12 Challenges and mitigation

Table 17. Challenges met so far and how we address them.

Challenge	Mitigation
False claims are rare: 6 to 15 on validation by the v2 LLM labels (50 by the automatic labels), while the primary metric needs about 300; Laya also has no false-claim head yet.	Run test passes until 300 false claims or 8 passes, decided before seeing any test result. Train a typed closing-message head. Plan to create training false claims by editing passed traces so their last needed call fails, which replay confirms.
The oracle's off-track label disagrees with the annotators (kappa 0.04 to 0.15), fails the proposal's 80% rule, yet it is set on 276 of the step head's 408 bad training calls (as "breaks task").	Not fixed yet. Retrain the step head without it, or on annotator-checked labels, and compare; meanwhile the step-check results in Table 12 are scored against labels that include this rule.
Annotators disagreed on turn ends (kappa 0.54 under guide v1, 0.61 under v2).	Ran the 200-step human check and rewrote the guide (v3). All-step agreement with the check is now 0.84 and 0.71 (past 0.7 only after reweighting). On turn ends it is not resolved: the annotators agree with each other at 0.61 to 0.62, B agrees with the check at 0.60 to 0.61, and false-claim agreement fell. A further guide revision and the test-sample check follow.
Laya as is is unusable: it blocks 87.6% of steps and truncates the step it judges.	Step first in the state, and a full finetune: AUROC 0.91 on calls.
A step judge cannot see omissions, which are about half of all failures.	Added the done check at turn end, and rollback in place of block and warn.
Little room on the 9B agent, and a noisy pass rate: oracle +4.2; 28 of 120 tasks flip between identical runs.	Three seeds per system, and a pilot on the weaker 4B agent (oracle +9.2). Report the per-step F1 claim separately from task accuracy.
A threshold chosen on 50 dev positives overfit: F1 0.36, against 0.53 at 0.5.	Choose thresholds by grouped cross-validation on train.
Compute: the 2080 Ti cards on Ada are slow, Granite timed out, and the directed runs with a large model cost too many GPU-hours.	Moved the 9B runs to a 48 GB RTX 6000 Ada on Turing, scored Granite on CPU, and dropped the Qwen3.8 27B directed runs.
BFCL quality: unsound answer keys, and the turn where the agent should ask is never scored.	Use the cleaned split and the empty-turn check; we plan to also report act-or-ask decision accuracy.

13 Changes from the proposal

This report is measured against the revised proposal (`proposal/proposal.md`).

Changes from the revised proposal to now:

- **Directed runs dropped.** The planned Qwen3.8 27B directed runs were dropped by team decision on 24 October to save GPU time. As a result, two step classes stay below Gate 1 (unneeded irreversible calls 217, made-up values at most 143); the step head pools them into one "bad step" class. The step head trains on pooled natural mistakes from two agent models, Qwen3.5 9B and Qwen3 8B. False-claim examples are planned to come from edited traces.
- **Natural training runs.** The Qwen3.5 9B ran its planned 3 passes (888 train tasks). The Qwen3.5 4B pass was replaced by the recorded Qwen3 8B steps.
- **Test-set baselines.** They ran with the earlier Qwen3 8B agent and judge, not the proposal's Qwen3.5 9B; they are to be rerun with the 9B for the task claim.
- **Test passes.** Up to 8 passes instead of 3 (the proposal allows a fourth as a fallback), stopping at 300 false claims; the stop rule is fixed before seeing any test result.
- **Automatic-label check.** The planned 350-step check of each automatic rule was not run as specified, and the step head still trains on the oracle off-track rule (Section 12).
- **Correction actions.** Block plus a fixed warning and a fresh doer became rollback-and-resample plus the done check, because block and warn did not help (Section 9).
- **A fourth question.** The done check at turn end (done or not done; a typed version is planned) covers the half of failures that a step judge cannot see.

- **Q3 folded in, heads binary.** The unneeded-irreversible-call question is one of the "bad step" labels of a single step head. Q1 and Q2 are not yet separate outputs, so the per-question comparison is still to come.
- **Annotator B.** GLM-5.3-Flash was used instead of GLM-5.3.
- **Hardware.** The 9B runs and the Laya finetunes ran on a 48 GB RTX 6000 Ada on Turing.

Changes from proposal v5 to the revised proposal:

- **Agent model.** Qwen3 8B became Qwen3.5 9B (43.3% against 25.8% on validation). Qwen3 8B remains the agent of the test-set baselines (Table 10).
- **Labelling.** Hand-labelling 2,000 to 3,000 test steps became two LLM annotators labelling every step, with a stratified 200-step human check and the decision rule of Section 10.
- **Directed-run model and reference judge.** Qwen3 32B became Qwen3.8 27B; Qwen3 14B became Gemma 4 12B, a different model family from the agent.

14 Timeline to the final submission

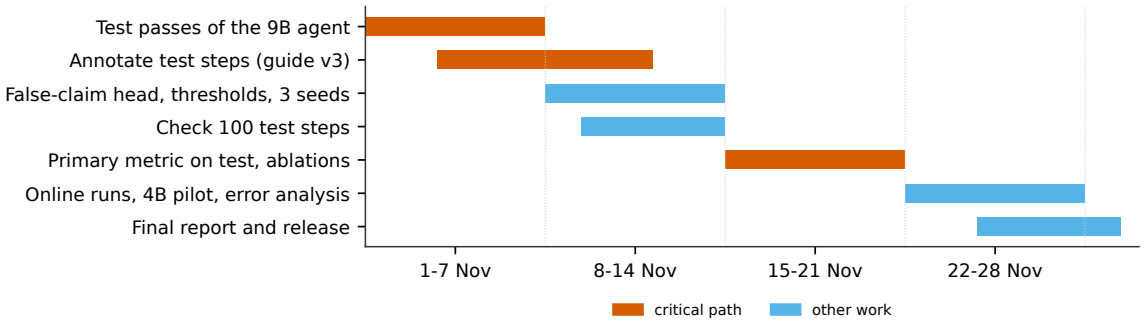


Figure 7. Plan to the final submission (weeks of November 2026).

Table 18. Work plan and owners.

Week	Work	Owner
31 Oct	Finish the two Phase 3 runs on Ada (training labels without "breaks task"; done-head threshold by cross-validation) and score them.	Swayam
1–7 Nov	Test passes of the 9B agent (about 0.5 GPU-hour each) until 300 false claims; LLM annotation of the test steps with guide v3.	Pavan
8–14 Nov	Typed closing-message head with a false-claim class; thresholds by grouped cross-validation; 3 seeds. Human check of 100 test steps (Pavan referees).	Pavan, Shuban; check: Swayam, Aaditya
15–21 Nov	Primary metric on test with paired bootstrap CIs and Holm correction; state-part and escalation-budget ablations; cost per judge.	Shuban, Swayam
22–28 Nov	Online test runs (no judge, Laya, SLM judge, oracle) with rollback and the done check; 4B pilot; error analysis of every miss and false alarm.	Pavan, Aaditya
By the deadline	Final report, release of the checked annotations, repository clean-up.	Everyone

Dates shift with the final deadline. The critical path is the test passes, then the test annotation, then the offline test evaluation.

15 Repository and reproduction

Repository: `https://github.com/ch-pavan/lma-grp-project/tree/main`

Table 19. Repository layout.

Path	Contents
`proposal/`	Proposal source, figures and PDF
`shuban_findings/`	Harness code (`code/`), Ada job scripts, test baselines, correction modes, off-the-shelf judges, Qwen3 8B finetune data
`analysis/`	Oracle and step labels, annotation scripts, Laya finetuning and evaluation, primary metric, write-ups
`human_check/`	Annotation guides v2 and v3, the two check samples, checkers' sheets and results
`runs/raw/`, `runs/data/`	Run outputs, labels, annotations and Laya datasets
`runs/turing/`	Job scripts for the 9B runs and Laya finetunes
`mid_submission/`	This report (LaTeX source), its figures and the build scripts
`docs/phase_plan.md`	Phase-wise plan from now to the final, with owners and done-tests
`pavan_gpu/`	GPU job scripts (Turing and Ada) for the running and Phase 4 jobs, and their results

```
sh runs/unpack_data.sh                                  rebuild runs/data from the repository
python analysis/primary_val.py                          primary metric on validation
python human_check/score.py human_check/val2 --guide v3 annotation check, fresh sample
python mid_submission/make_figs.py                      redraw the figures
python mid_submission/build_report.py                   rebuild this PDF
```